Problem zero (simulating a preceding fragment)

Some preceding text so the fragment is not the first thing in the document.

Ex. 3.1 — Loading the OS and one program without an MMU

The constraint. With no MMU there is no translation step: the address the CPU puts on the bus is the physical address. The program is linked to begin at 0 and to run over a contiguous range, so physical address 0 and the bytes above it must belong to the application, and the OS cannot have them.

The layout. Load the OS at the high end of RAM, growing downward from the physical top, and the application at physical 0, growing upward: text, then data and bss, then the heap, with the stack at the top of the user region growing down (Figure 1). The program owns the whole of $[0, F)$, where F is the bottom of the OS, so both of the CPU's assumptions hold with no translation and no relocation: the loader copies the executable to address 0 byte for byte. Running a different program overwrites $[0, F)$; the OS never moves.

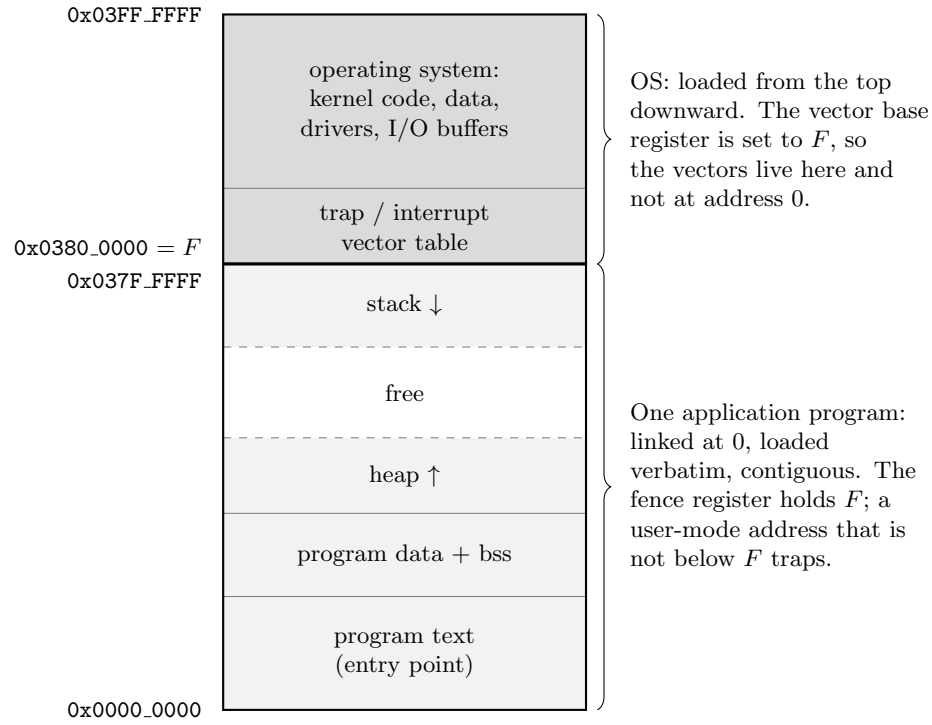


Figure 1: Memory map for a single-program system with no MMU. Physical memory is 64 MiB and the OS occupies the top 8 MiB. Addresses increase upward.

Address 0 is not usually free. Most CPUs fetch trap and interrupt vectors from fixed low addresses: the 68000's table at `0x000`–`0x3FF`, x86 real mode's IVT at `0x00000`–`0x003FF`, ARM's vectors at 0. That collides with the layout, so I require a CPU with a vector base register (68010 VBR, ARM VBAR, RISC-V `mtvec`) and set it to F. Such a register is consulted only when an exception is dispatched and leaves ordinary loads and stores untouched, so it is not an MMU. On

a CPU that hard-wires its vectors at 0 the assumption cannot be met: the vectors keep the low addresses and the program has to be relocated when it is loaded. Only the reset vector cannot be moved in principle, and it is served from boot ROM.

The boundary. No MMU also means no permission check per access, so nothing above stops the application from storing into the OS. I therefore allow one fence (limit) register holding F: in user mode the CPU traps any address that is not below F. A comparison adds nothing to the address, so this is still not translation; a *base* register, added to every address, is where an MMU begins.

Problem two (simulating a following fragment)

Some following text.